

MGC Module

The diagram illustrates the architecture of the MGC Module, which processes two input feature maps, F_M and F_H , to produce output feature maps $\tilde{F}_M$ and $\tilde{F}_H$.

Input Processing:

- Input F_M is downsampled by a factor of 8 ($\downarrow 8$).
- Input F_H is downsampled by a factor of 8 ($\downarrow 8$).

Processing Path for F_M :

- The downsampled F_M is processed by an MCA block using V_M^1 , K_M^1 , and Q_M^1 to produce C_M^1 .
- C_M^1 is upsampled by a factor of 2 ($\times 2$) and added ($\oplus$) to the original downsampled F_M .
- The result is processed by another MCA block using V_M^2 , K_M^2 , and Q_M^2 to produce C_M^2 .
- C_M^2 is upsampled by a factor of 2 ($\times 2$) and added ($\oplus$) to the previous result.
- The final result for F_M is $\tilde{F}_M$.

Processing Path for F_H :

- The downsampled F_H is processed by an MCA block using V_H^1 , K_H^1 , and Q_H^1 to produce C_H^1 .
- C_H^1 is upsampled by a factor of 2 ($\times 2$) and added ($\oplus$) to the original downsampled F_H .
- The result is processed by another MCA block using V_H^2 , K_H^2 , and Q_H^2 to produce C_H^2 .
- C_H^2 is upsampled by a factor of 2 ($\times 2$) and added ($\oplus$) to the previous result.
- The final result for F_H is $\tilde{F}_H$.

Legend:

- $\rightarrow$ Average pooling
- $\rightarrow$ UpSample
- $\oplus$ Pixel-wise Addition

中文题目: 高光谱与多光谱图像融合的自适应专家学习

所属单位：深圳大学

核心速览：提出一种自适应专家学习框架（AELF），通过动态建模不同区域的模态主导性并相应调整融合策略，以解决高光谱图像（HSI）和多光谱图像（MSI）融合中现有方法依赖静态融合策略、忽略区域变异性的问题。

论文内容详细解读:

<https://mp.weixin.qq.com/s/WT5TwGfQz1MuS930h6fLcw>

```

MGCM(
  (in_x): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (in_y): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pools_x): ModuleList(
    (0): AvgPool2d(kernel_size=8, stride=8, padding=0)
    (1): AvgPool2d(kernel_size=4, stride=4, padding=0)
    (2): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
  (pools_y): ModuleList(
    (0): AvgPool2d(kernel_size=8, stride=8, padding=0)
    (1): AvgPool2d(kernel_size=4, stride=4, padding=0)
    (2): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
  (convs_x): ModuleList(
    (0-2): 3 x Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  )
  (convs_y): ModuleList(
    (0-2): 3 x Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  )
  (attns): ModuleList(
    (0-2): 3 x MutCrossAttention(
      (mca): CrossTransAttention(
        (kv): Conv2d(64, 128, kernel_size=(1, 1), stride=(1, 1))
        (q): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1))
        (kv_dwconv): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=128)
        (q_dwconv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=64)
        (project_out): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1))
      )
    )
  )
  (relu): GELU(approximate='none')
  (sum_x): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (sum_y): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
)
Input fm shape: torch.Size([1, 64, 32, 32])
Input fh shape: torch.Size([1, 64, 32, 32])
Output fm shape: torch.Size([1, 64, 32, 32])
Output fh shape: torch.Size([1, 64, 32, 32])

```

哔哩哔哩/微信公众号：AI缝合术，独家整理！